

// Jivops

Azure Container Apps

Guía completa — 20 lecciones

Fundamentos · Red y escalado · Seguridad y datos · Operación

Contenido

Módulo 1: Fundamentos

- Lección 01 — Qué es Azure Container Apps
- Lección 02 — Entornos
- Lección 03 — Aplicaciones y contenedores
- Lección 04 — Revisiones
- Lección 05 — Perfiles de carga: Consumption y Dedicated

Módulo 2: Red, escalado y despliegue

- Lección 06 — Ingress
- Lección 07 — Redes y VNet
- Lección 08 — Escalado con KEDA
- Lección 09 — División de tráfico y blue/green
- Lección 10 — Dominios personalizados y certificados

Módulo 3: Seguridad y datos

- Lección 11 — Identidades gestionadas
- Lección 12 — Secretos
- Lección 13 — Almacenamiento
- Lección 14 — Dapr
- Lección 15 — Jobs

Módulo 4: Operación y costes

- Lección 16 — Observabilidad
- Lección 17 — Infraestructura como código
- Lección 18 — Optimización de costes
- Lección 19 — Resolución de problemas
- Lección 20 — Proyecto final: plataforma de microservicios en Container Apps

Módulo 1: Fundamentos

Lección 01 — Qué es Azure Container Apps

Un servicio serverless de contenedores de Azure para ejecutar microservicios, APIs, trabajos y procesos en segundo plano sin gestionar el clúster Kubernetes subyacente, con escalado automático (incluido a cero), ingress, revisiones y Dapr integrados.

No gestionas Kubernetes, escala a cero y reduce la operación, ideal para microservicios y APIs.

Lección 02 — Entornos

El límite de aislamiento y de red de un conjunto de aplicaciones: comparten red virtual, destino de logs y configuración, y las apps del mismo entorno pueden comunicarse de forma privada.

Cuando necesitas aislamiento de red o gestión, por ejemplo separar producción de pruebas o equipos.

Lección 03 — Aplicaciones y contenedores

Con uno o más contenedores (imagen, CPU, memoria, variables, sondas de salud), sidecars e init containers opcionales, y la configuración de ingress, escalado y secretos; las imágenes pueden venir de Azure Container Registry u otro registro.

Para saber si el contenedor está vivo o listo y reiniciarlo o dirigirle tráfico según corresponda.

Lección 04 — Revisiones

Una instantánea inmutable de la versión de la aplicación; cambiar la plantilla (imagen, recursos, escalado) crea una revisión nueva, y en modo de revisión única solo una está activa mientras que en modo múltiple pueden convivir varias.

Permite probar una versión nueva con parte del tráfico y volver atrás si falla.

Lección 05 — Perfiles de carga: Consumption y Dedicated

Opciones de capacidad del entorno: Consumption, serverless que se factura por uso y escala a cero, y perfiles Dedicated con tamaños de máquina fijos (incluido GPU) para cargas que necesitan más recursos o rendimiento predecible.

Para cargas que necesitan más CPU o memoria, GPU o rendimiento predecible.

Módulo 2: Red, escalado y despliegue

Lección 06 — Ingress

Habilitando ingress con un puerto de destino: externo (accesible desde internet con un FQDN) o interno (solo dentro del entorno o la red virtual); el tráfico pasa por un proxy gestionado con HTTPS, HTTP/2 y balanceo entre réplicas.

Para microservicios o backends que no deben ser públicos y solo los llaman otras apps.

Lección 07 — Redes y VNet

Creando el entorno en una subred de una VNet propia, con ingress interno o externo, y controlando la salida con grupos de seguridad de red y rutas definidas por el usuario; los endpoints privados permiten acceso sin exponer a internet.

Para acceder a recursos privados como bases de datos u on-premises y aislar el tráfico de internet.

Lección 08 — Escalado con KEDA

Con reglas de escalado basadas en KEDA: HTTP (solicitudes concurrentes), TCP, colas (Service Bus, Storage Queue), Event Hubs, CPU o memoria y métricas personalizadas; con un mínimo y máximo de réplicas, pudiendo llegar a cero.

Ahorra coste cuando no hay carga, pero la primera petición sufre un arranque en frío con más latencia.

Lección 09 — División de tráfico y blue/green

En modo de revisión múltiple se reparte el porcentaje de tráfico entre revisiones, se prueba la nueva con una parte, se sube el porcentaje y se desactiva la anterior; se puede revertir devolviendo el tráfico a la revisión antigua.

Limita el riesgo: si hay errores solo afectan a pocos usuarios y se puede revertir rápido.

Lección 10 — Dominios personalizados y certificados

Añadiendo el dominio a la app, validando la propiedad con registros DNS (CNAME o TXT) y asociando un certificado, que puede ser gestionado por Azure o propio, de modo que el tráfico se sirve con TLS en tu nombre.

Que los registros DNS de validación estén bien creados y propagados.

Módulo 3: Seguridad y datos

Lección 11 — Identidades gestionadas

Para que la aplicación se autentique ante otros servicios de Azure (Key Vault, Storage, ACR, bases de datos) sin contraseñas ni claves en el código: Azure le da una identidad, ya sea asignada por el sistema o por el usuario, y se le conceden roles con RBAC.

No hay secretos que filtrar ni rotar: Azure gestiona las credenciales automáticamente.

Lección 12 — Secretos

Guardándolos como secretos de la app y referenciándolos en variables de entorno, o mejor con referencias a Azure Key Vault resueltas mediante identidad gestionada, para no escribirlos en la imagen ni en la plantilla.

Centraliza los secretos con rotación y auditoría, sin copiarlos en cada app.

Lección 13 — Almacenamiento

Almacenamiento efímero del contenedor y de la réplica, montaje de Azure Files para datos compartidos y persistentes, y servicios externos como Blob Storage o bases de datos para el estado; los contenedores deben ser sin estado.

Para poder escalar y reemplazar réplicas sin perder datos, que se guardan fuera.

Lección 14 — Dapr

Un runtime de sidecar con APIs para tareas habituales de microservicios: invocación entre servicios, publicación y suscripción de mensajes, estado, bindings y secretos, desacoplando la app de la infraestructura concreta.

Desacopla la app del broker concreto y permite cambiarlo con poco cambio en el código.

Lección 15 — Jobs

Contenedores que se ejecutan hasta terminar en lugar de servir peticiones continuamente: manuales, programados con expresión cron o basados en eventos; se usan para procesamiento por lotes, migraciones y tareas periódicas.

Cuando la tarea termina, como procesos por lotes, migraciones o tareas periódicas.

Módulo 4: Operación y costes

Lección 16 — Observabilidad

Enviando los logs de consola y del sistema a Log Analytics (o a otros destinos) y consultándolos con KQL, usando métricas de Azure Monitor (réplicas, solicitudes, CPU, memoria), Application Insights para trazas y alertas.

Alertas por errores, reinicios, latencia alta y número de réplicas anómalo.

Lección 17 — Infraestructura como código

Con Bicep, plantillas ARM o Terraform que describen entorno, apps, revisiones, secretos, ingress y escalado, y con la CLI az containerapp para operaciones puntuales; el código se versiona y se despliega desde un pipeline.

Reproducibilidad, versionado y revisión de cambios, con entornos idénticos.

Lección 18 — Optimización de costes

Ajustando CPU y memoria al uso real, escalando a cero cuando se pueda, limitando las réplicas máximas, eligiendo Consumption o Dedicated según el patrón de uso, y revisando la retención de logs.

Cuando la latencia importa o el tráfico es constante, por el arranque en frío.

Lección 19 — Resolución de problemas

El estado y los eventos de la revisión, los logs de consola y del sistema, que la imagen se pueda descargar (permisos al registro), que el puerto del contenedor coincida con el del ingress y que las sondas de salud respondan.

Falta de permisos o credenciales hacia el registro, por ejemplo sin identidad con rol AcrPull.

Lección 20 — Proyecto final: plataforma de microservicios en Container Apps

Entorno en VNet con ingress externo solo para el frontal e interno para el resto, imágenes en ACR, identidades gestionadas y Key Vault, escalado KEDA, revisiones con reparto de tráfico, Dapr o Service Bus para mensajería, jobs para lotes, Log Analytics con alertas e infraestructura en Bicep.

Crear una revisión, enviarle un porcentaje de tráfico, probar y subir o hacer rollback si falla.